

NEUR 540 • GRADUATE SEMINAR • FALL 2026

Computational Neuroscience: from spikes to circuits

A twelve-week graduate course modelling how neurons compute, individually and together.

What this first session covers.

- 01 What the course is about
- 02 The twelve-week arc
- 03 What you will be able to do by the end
- 04 Logistics, grading, and what to do this week

This course builds a working brain model, one layer at a time.

We start from a single neuron's electrical behaviour and end with networks that learn. Every week adds a layer: from the membrane equation, to spiking models, to populations, to plasticity. By December you will have coded each one yourself.

Who this course is for, and what it assumes.

This is a graduate seminar for students with calculus, basic linear algebra, and some Python. We assume no prior neuroscience — the biology is taught alongside the math. If you can integrate a differential equation and write a for-loop, you are ready.

- Prerequisites: calculus, linear algebra, intro Python.
- No prior neuroscience required.

TWELVE WEEKS • THREE MODULES

How the course unfolds across the term.

MODULE	WEEKS 1–4	WEEKS 5–8	WEEKS 9–12
	SINGLE NEURON	POPULATIONS	LEARNING
Theory	✓ Membrane & cable	– Network dynamics	○ Synaptic plasticity
Coding labs	✓ Integrate-and-fire	– Recurrent networks	○ Learning rules
Readings	✓ Hodgkin–Huxley	– Attractor models	○ Predictive coding

✓ Shipped – In flight ○ Planned

By the end of the course, you will be able to:

01

Model a single neuron

Derive and simulate the membrane equation and the integrate-and-fire model from first principles.

02

Build a network

Implement recurrent populations and explain how circuit dynamics give rise to memory and decisions.

03

Implement learning

Code biologically grounded plasticity rules and show how a network learns from experience.

04

Read the literature critically

Evaluate a computational neuroscience paper's model, assumptions, and claims.

How you are assessed: code more than exams.

Grades come mostly from the weekly coding labs, where you build the models we derive in lecture. A midterm tests the theory, and a final project lets you model a system of your own choosing. There is no final exam.

- Weekly labs: 50%. Midterm: 20%. Final project: 30%.
- No final exam — the project is the capstone.

The final project: model one phenomenon end to end.

In the last month you will pick a neural phenomenon — a visual illusion, a decision, a memory effect — and build a computational model that reproduces it. You will present it in week twelve. Past students have modelled birdsong learning, place cells, and binocular rivalry.

- Pick a phenomenon by week eight; build it by week twelve.
- A short talk plus your code is the deliverable.

Course policies, briefly.

Labs are due Fridays at 5pm; two no-questions late days are yours to spend across the term. Collaboration on labs is encouraged, but the code and the write-up must be your own. All materials are open and posted to the course site.

- Two flexible late days, no explanation needed.
- Discuss freely; submit your own work.

What to do before next week.

- ☒ Enroll and join the course forum.
- ☐ Install the Python environment from the setup guide.
- ☐ Read Chapter 1 — the membrane equation.
- ☐ Run the warm-up notebook and post one question.
- ☐ Skim the syllabus and flag any scheduling conflicts.

Office hours and how to get unstuck.

I hold office hours Tuesdays and Thursdays, and the TA runs a lab clinic before each deadline. The fastest help is the forum — post your error and someone, often a classmate, will answer within hours. Do not suffer alone with a bug.

- Office hours Tue/Thu; TA clinic before each lab is due.
- Post errors to the forum first — it is the fastest channel.

INSTRUCTOR • PROF. HANA YOSHIDA • COURSE SITE AT [NEUR540.EXAMPLE](#)

By December, you will have built
a brain that learns — line by line.